

Fiche de révision — Administration & Automatisation Azure (TP3)

Cours magistral « *Administration Cloud et Automatisation Azure* » — Bloc 4, Mastère Dev Manager Full Stack. Thèmes : **Azure CLI, Bash, Python (SDK), Azure Monitor, sécurité, FinOps**. Cas fil rouge : exploitation de **ShopEasy**.

1. Du déploiement à l'exploitation

Provisionner n'est pas exploiter. Le provisionnement (TP1/TP2) crée les ressources. L'**exploitation** commence *après* le déploiement et répond à d'autres questions : les ressources sont-elles disponibles ? consomment-elles trop ? exposent-elles des ports sensibles ? sont-elles taguées ? les coûts sont-ils conformes ?

Les 7 familles d'actions d'exploitation :

Famille	Objectif	Exemples Azure
Inventaire	Savoir ce qui existe	<code>az resource list</code> , export CSV, tags
Administration	Modifier / piloter	start/stop VM, resize, update tags
Supervision	Observer l'état	Azure Monitor, metrics, logs, alertes
Automatisation	Réduire le manuel	Bash, Python, scripts planifiés
Sécurité	Réduire l'exposition	RBAC, NSG, diagnostic, audit
FinOps	Maîtriser les coûts	budgets, tags, arrêt VM, ressources orphelines
Documentation	Transmettre / pérenniser	rapport d'exploitation, runbooks

Cas entreprise. Une VM peut être parfaitement déployée par Terraform **et** mal exploitée : pas de tag propriétaire, SSH ouvert à Internet, aucune alerte CPU, aucun suivi de coût. L'exploitation, c'est traiter exactement ces angles morts au quotidien.

2. Azure Resource Manager (ARM) & modèle de ressources

ARM est la couche de gestion d'Azure. Hiérarchie : **Tenant** (annuaire Entra ID) → **Subscription** (facturation + droits) → **Resource Group** (regroupe une app/projet/env) → **Resources** → **Tags** (métadonnées).

Une équipe d'exploitation travaille **par périmètre** (application, environnement, équipe, centre de coût), pas ressource par ressource. Le Resource Group n'est pas qu'un contenant technique : c'est une **unité d'exploitation, de gouvernance et de nettoyage**.

Cas entreprise. « Quelles ressources appartiennent à ShopEasy ? Lesquelles sont en prod ? Qui en est responsable ? Lesquelles peuvent s'arrêter le soir ? » → ces questions opérationnelles ne se répondent vite que si RG + tags sont bien organisés.

3. Azure CLI : l'outil central d'administration

Outil en ligne de commande pour gérer Azure (poste local, Cloud Shell ou pipeline CI/CD). **Préféré au portail** pour l'exploitation car les commandes sont **rejouables, documentables, scriptables et automatisables** — là où le portail favorise le clic manuel non standardisé et laisse peu de traces.

Commandes de base :

```
az login
az account show
az account set --subscription "<id>"
az group list -o table
az resource list -g rg-shopeasy-dev -o table
```

Formats de sortie (-o) — à choisir selon le besoin :

Format	Usage
json	Traitement programmatique (scripts, Python, jq)
table	Lecture humaine rapide
tsv	Extraction dans une variable Bash
yaml	Lecture structurée
none	Masquer la sortie dans un script

--query + JMESPath = filtrer/transformer le JSON :

```
az vm list -g rg-shopeasy-dev \
  --query "[].{name:name, size:hardwareProfile.vmSize, location:location}" -o table

az resource list -g rg-shopeasy-dev \
  --query "[?tags.Environment=='dev'].{name:name,type:type}" -o table
```

Cas entreprise. Un audit demande « la liste des VM avec leur taille et leur région ». En CLI + **--query**, c'est une commande rejouable et copiable dans un rapport. En portail, c'est du clic non reproductible.

4. Inventaire & tags

Inventorier = la base de l'exploitation : avant de surveiller/sécuriser/optimiser, il faut savoir ce qui existe (combien de ressources, quels types, où, taguées ou non, lesquelles ont une IP publique,

lesquelles sont démarrées).

Tags utiles :

Tag	Exemple	Utilité
Environment	dev / rec / prod	Filtrer par environnement
Owner	ops-team	Identifier le responsable
CostCenter	CC-2045	Affecter les coûts
Application	ShopEasy	Relier à l'application
Criticality	low / medium / high	Prioriser la surveillance
AutoShutdown	true / false	Automatiser l'arrêt des VM

Cas entreprise. Un environnement sans tags est ingérable : impossible de savoir qui paie, qui est responsable, quoi arrêter. Les tags sont simples à poser mais **très puissants** pour FinOps, audit, inventaire et automatisation.

5. Bash pour automatiser l'exploitation

Bash enchaîne des commandes, factorise des variables, contrôle les erreurs et produit des fichiers de sortie (inventaire récurrent, rapport, start/stop, contrôle de tags...).

Structure minimale d'un script fiable :

```
#!/usr/bin/env bash
set -euo pipefail          # stop si erreur / variable non définie / pipe échouée
RG="rg-shopeasy-dev"
OUT_DIR="./out"
mkdir -p "$OUT_DIR"
az resource list -g "$RG" \
  --query "[].{name:name,type:type,location:location}" \
  -o table | tee "$OUT_DIR/inventory.txt"
```

Bonnes pratiques : `set -euo pipefail`, variables centralisées, répertoire de sortie, messages explicites, **commandes idempotentes** (relançables sans effet de bord), journalisation des contrôles.

Cas entreprise. Un script d'inventaire lancé chaque matin produit `inventory-AAAAMJJ.txt`. Grâce à `set -euo pipefail`, il s'arrête net si une commande échoue plutôt que de générer un rapport faux et trompeur.

6. Administration des VM

Cycle de vie : créée → démarrée → arrêtée → désallouée → redémarrée → supprimée.

⚠ **stop vs deallocate (question d'examen classique)** : - `az vm stop` → arrête la VM mais la **capacité de calcul reste réservée** → **continue de facturer le compute**. - `az vm deallocate` → **libère la capacité de calcul** → plus de coût compute (mais **les disques restent facturés**).

```
az vm get-instance-view -g $RG -n $VM --query "instanceView.statuses[].displayStatus" -o tsv
az vm deallocate -g $RG -n $VM # pour le FinOps : arrêter les VM dev
az vm run-command invoke -g $RG -n $VM --command-id RunShellScript \
  --scripts "uptime && df -h && systemctl status nginx --no-pager"
```

Cas entreprise. Une équipe « arrête » ses VM dev via l'OS le soir, mais la facture compute ne baisse pas. Le réflexe correct : **deallocate** (ou un planning d'auto-shutdown). Confondre les deux peut coûter des milliers d'€/an.

7. Stockage des rapports d'exploitation

Les rapports (inventaires, exports CSV, contrôles) doivent être **conservés et historisés** dans un Storage Account / conteneur Blob pour le suivi dans le temps.

```
az storage blob upload --account-name stshopeasyreports001 \
  --container-name reports --name "inventory-$(date +%Y%m%d).txt" \
  --file ./out/inventory.txt --auth-mode login --overwrite true
```

Cas entreprise. Une équipe mature ne se contente pas d'exécuter des commandes : elle **conserve les preuves** des contrôles. En cas d'incident ou d'audit, on peut montrer l'état de l'environnement à une date donnée.

8. Azure Monitor & observabilité

L'observabilité repose sur 3 familles d'information : - **Métriques** : valeurs numériques dans le temps (CPU, réseau, requêtes...). - **Logs** : événements détaillés des ressources/apps/systèmes. - **Alertes** : règles qui préviennent quand un seuil/condition est atteint.

Métriques utiles ShopEasy : VM (CPU, réseau, disque, disponibilité), Load Balancer (santé des probes), Storage (transactions, latence), Azure SQL (DTU/vCore, connexions), NSG (flux refusés).

```
VM_ID=$(az vm show -g $RG -n vm-shopeasy-web-01 --query id -o tsv)
az monitor metrics alert create -g $RG -n "alert-high-cpu-vm-web-01" \
  --scopes "$VM_ID" --condition "avg Percentage CPU > 80" \
  --window-size 5m --evaluation-frequency 1m
```

Bonnes pratiques d'alerting : éviter le trop-plein d'alertes, **relier chaque alerte à une action attendue**, documenter la procédure, adapter les seuils par environnement. *Une alerte sans procédure n'est que du bruit.*

Cas entreprise. Une alerte CPU qui se déclenche sans qu'on sache quoi faire est ignorée au bout d'une semaine (« fatigue d'alerte »). La bonne alerte dit : *quoi vérifier, qui contacter, quelle action, quelle urgence.*

9. FinOps opérationnel

Le coût est un **signal d'exploitation** : VM oubliées, disques orphelins, IP publiques inutilisées, stockage non nettoyé, ressources surdimensionnées = coûts invisibles.

Actions FinOps simples : taguer (CostCenter/Owner/Environment), **désallouer les VM dev hors usage**, supprimer les ressources orphelines, dimensionner (rightsizing), poser des budgets, analyser via Cost Management.

```
# disques orphelins (non rattachés à une VM)
az disk list -g $RG --query "[?managedBy==null].{name:name,size:diskSizeGb}" -o table
# IP publiques (exposition + coût)
az network public-ip list -g $RG --query "[].{name:name,ip:ipAddress}" -o table
```

Cas entreprise. Après plusieurs déploiements/destructions partielles, il reste 12 disques orphelins facturés pour rien. Une requête `managedBy==null` les détecte → suppression → économie immédiate. *Le FinOps est une discipline d'exploitation, pas seulement financière.*

10. Sécurité d'exploitation

La sécurité n'est pas figée au design : une archi sûre **dérive** avec le temps (règles modifiées, droits excessifs, logs désactivés).

Point de contrôle	Question	Remédiation
RBAC	Qui a accès au RG ?	Limiter aux rôles nécessaires
Ports exposés	SSH/RDP ouverts à Internet ?	Restreindre par IP ou Azure Bastion
NSG	Règles trop larges ?	Réduire sources/destinations
Secrets	Secrets dans les scripts ?	Key Vault / variables sécurisées
Logs	Activité tracée ?	Activer diagnostic + Activity Log
Tags	Responsable identifié ?	Ajouter Owner + Application

⚠ Une règle entrante `0.0.0.0/0` vers **SSH (22)** ou **RDP (3389)** est une **anomalie** hors labo strictement contrôlé.

Cas entreprise. Un `az network nsg rule list` régulier détecte une règle SSH ouverte au monde entier laissée après un dépannage. On la restreint à l'IP admin (ou Bastion) avant qu'elle ne soit exploitée par un scan automatisé.

11. Python & SDK Azure

Quand passer de Bash à Python ? Quand le besoin grandit : traitement structuré, appels API, génération de rapports, enrichissement de données, intégration, tests, orchestration complexe.

Bibliothèque	Rôle
<code>azure-identity</code>	Authentification (<code>DefaultAzureCredential</code>)
<code>azure-mgmt-resource</code>	Resource groups & ressources ARM
<code>azure-mgmt-compute</code>	VM & compute
<code>azure-mgmt-monitor</code>	Monitoring

```
from azure.identity import DefaultAzureCredential
from azure.mgmt.resource import ResourceManagementClient
client = ResourceManagementClient(DefaultAzureCredential(), "<subscription-id>")
for r in client.resources.list_by_resource_group("rg-shopeasy-dev"):
    print(r.name, r.type, r.location)
```

Cas entreprise. Un inventaire simple → Bash suffit. Mais produire un **rapport consolidé** (plusieurs subscriptions, mise en forme, envoi par mail, croisement avec un CMDB) → Python + SDK est le bon outil. *Azure CLI est souvent le meilleur point de départ ; Python industrialise.*

12. Le rapport d'exploitation (le livrable clé)

La valeur du TP n'est pas la liste des commandes, mais le **rapport** qui prouve que l'environnement a été compris, contrôlé et amélioré.

Structure recommandée : 1) contexte/périmètre · 2) inventaire · 3) état des VM/services · 4) tags & gouvernance · 5) réseau & sécurité · 6) supervision & alertes · 7) FinOps · 8) risques résiduels · 9) recommandations priorisées.

Il doit parler à **deux publics** : l'équipe technique (qui rejoue les commandes) **et** la DSI (qui comprend risques, coûts, priorités).

Exemple de formulation exploitable : ❌ « La VM marche. » → ✅ « La VM `vm-shopeasy-web-01` est en état *running*, répond au test HTTP, mais n'a pas de tag `CostCenter`. Recommandation : ajouter ce tag et créer une alerte CPU à 80 %. »

13. Architecture d'exploitation cible (synthèse)

Besoin	Outil / service	Résultat attendu
Inventaire	Azure CLI	Liste fiable des ressources

Besoin	Outil / service	Résultat attendu
Automatisation simple	Bash	Scripts relançables
Automatisation avancée	Python SDK	Rapport structuré
Surveillance	Azure Monitor	Métriques + alertes
Historisation	Azure Storage	Rapports conservés
Coûts	Cost Management	Budget + optimisation
Sécurité	RBAC, NSG, logs	Risques réduits

Les réflexes à retenir pour le TP3

1. **CLI plutôt que portail** : rejouable, documentable, scriptable.
2. **--query (JMESPath)** pour filtrer/mettre en forme les sorties.
3. **deallocate ≠ stop** : seul **deallocate** coupe le coût compute.
4. **Tout est tagué** (Owner, Environment, CostCenter...) — sinon ingérable.
5. **Une alerte sans procédure = du bruit.**
6. **Scripts fiables** : **set -euo pipefail**, idempotents, sorties conservées.
7. **Pas de 0.0.0.0/0 vers SSH/RDP.**
8. Le **rapport d'exploitation** est le vrai livrable (technique + DSI).

Glossaire express

Terme	Définition
Azure CLI	Outil ligne de commande pour gérer Azure
JMESPath	Langage de requête utilisé par --query
ARM	Azure Resource Manager, couche de gestion des ressources
deallocate	Libère la capacité compute d'une VM (coupe le coût compute)
Azure Monitor	Collecte, analyse et alerting sur métriques/logs
Metric alert	Alerte déclenchée par une condition sur une métrique
Action group	Actions déclenchées par une alerte (email, webhook...)
RBAC	Contrôle d'accès basé sur les rôles
NSG	Règles réseau filtrant le trafic
FinOps	Discipline de gouvernance/optimisation des coûts cloud

Note déploiement (Azure for Students). Comme aux TP1/TP2, utiliser `swedencentral` (et non `francecentral`) et une taille VM disponible (`Standard_B2ts_v2`) si tu recrées des ressources.